

Q1)

Essentially, we are trying to cut the wire to the smallest piece, which is 1 meter. We do this by dividing the wire into 2 pieces and stack the remaining pieces together. Cut these pieces again and stack them again. Do this until each part is 1 meter long.

$T(n) = 1 \rightarrow$ best case

$T(n) = T(n/2) + 1 \rightarrow$ worst case

Using master theorem:

$a=1, b=2, f(n)=n^0, c=0$ $n^{\log_b a} = n^{\log_2 1} = n^0 = 1 \rightarrow$ since $n^0 = 1 \rightarrow$ case 2

$$\begin{aligned} T(n) &= \theta(n^{\log_b a} \cdot \log n) \\ &= \theta(1 \cdot \log n) = \theta(\log n) \end{aligned}$$

Q2)

We are recursively dividing the array into 2 equal parts and update the max and the min values of the whole array using recursion, with the predetermined minimum and maximum variables sent to the function. These are minus infinity and plus infinity. The base condition is when there is only one element left.

Since we are dividing the array into two parts;

$T(n) = 1 \rightarrow$ best case

$T(n) = T(n/2) + 1 \rightarrow$ worst case

Using master theorem:

$a=1, b=2, f(n)=n^0, c=0$ $n^{\log_b a} = n^{\log_2 1} = n^0 = 1 \rightarrow$ since $n^0 = 1 \rightarrow$ case 2

$$\begin{aligned} T(n) &= \theta(n^{\log_b a} \cdot \log n) \\ &= \theta(1 \cdot \log n) = \theta(\log n) \end{aligned}$$

Q3)

Decrease and conquer style of this question is done by Quick Select algorithm, which is somewhat related to Quick sort. Unlike quicksort, quick select only recurs that contains the kth smallest element. We pick a pivot element and move that to its correct position and partition the remaining array. Since we are not trying to do a literal quicksort, we won't have to completely use it, but stop at where pivot is the smallest.

$T(n) = n \rightarrow$ runs through the array once $\rightarrow$ best case

$T(n) = T(n-1) + n \rightarrow$ worst case

$$T(n) = T(n-1) + n$$

$$T(n-1) = T(n-2) + n-1$$

$$T(n-2) = T(n-3) + n-2$$

...

$$T(3) = T(2) + 3$$

$$T(2) = T(1) + 2$$

$$T(1) = 1$$

$$\begin{aligned} T(n) &= n + (n-1) + (n-2) + \dots + 3 + 2 + 1 \\ &= n \cdot (n-1) / 2 \rightarrow T(n) = O(n^2) \end{aligned}$$

Proving:

Base case: $T(1) = 1 \cdot (2) / 2 = 1$

Assume $= T(n-1) = (n-1) \cdot n / 2 \rightarrow$ true

$$T(n) = T(n-1) + n$$

$$T(n) = \frac{(n-1) \cdot n}{2} + n = \frac{n(n+1)}{2}$$

$$\frac{n^2 - n + 2n}{2} = \frac{n^2 + n}{2} = \frac{n(n+1)}{2}$$

Q4)

Since we want a divide and conquer approach, we must use some form of sorting algorithm that divides itself. Most notably, mergesort. We count the number of inversions when 2 halves of the array are merged. We set them 2 indexes i and j. If one is greater than other, there are mid-i inversions. Since left and right is already sorted, remaining elements in the left will be greater than right of the middle.

$$T(n) = 2T(n/2) + n$$

Master Theorem:

$$a=2, b=2, f(n)=n, c=1 \quad n^{\log_b a} = n^{\log_2 2} = n^1 = n \rightarrow \text{since } n^1=n \rightarrow \text{case 2}$$

$$\begin{aligned} T(n) &= \theta(n^{\log_b a} \cdot \log n) \\ &= \theta(n \cdot \log n) = \theta(n \cdot \log n) \end{aligned}$$

Q5)

Brute Force: We just multiply it with itself, base number of times. ($4^5 = 4.4.4.4.4$)

$$\sum_{i=1}^n 1 = n \rightarrow T(n) = O(n)$$

Divide and Conquer: Let's assume we want to compute 2^8 . Rather than multiplying with itself, we square each time (or take the nth exponential). Thus, result is calculated like this: $((a^n)^n)^n$

As you can see, it does only 3 operations, instead of 8. $3 = \log(8)$.

Using master theorem:

$a=1, b=2, f(n)=n^0, c=0$ $n^{\log_b a} = n^{\log_2 1} = n^0 = 1 \rightarrow \text{since } n^0 = 1 \rightarrow \text{case 2}$

$$\begin{aligned} T(n) &= \theta(n^{\log_b a} \cdot \log n) \\ &= \theta(1 \cdot \log n) = \theta(\log n) \end{aligned}$$